

+ for std::vector concatenation

- Document number: P0473R0
- Date: 2016-10-13
- Reply-to: David Sankel <dsankel@bloomberg.net>
- Audience: Library Evolution

Abstract

before	after
<pre>std::vector<int> all_names = get_names_from_a(); std::vector<int> tmp = get_names_from_b(); all_names.insert(all_names.end(), std::make_move_iterator(tmp.begin()), std::make_move_iterator(tmp.end())); tmp = get_names_from_c(); all_names.insert(all_names.end(), std::make_move_iterator(tmp.begin()), std::make_move_iterator(tmp.end()));</pre>	<pre>const std::vector<int> all_names = get_names_from_a() + get_names_from_b() + get_names_from_c();</pre>

Concatenating vectors is an extremely common operation yet it currently require complicated code. This paper is an attempt to remedy this situation. We propose overloading the + and += operators for vector to abstract away the move iterators, better show intent, and allow more code to be const-correct.

Performance Implications

Prior to C++11, functions producing std::vector objects would typically take in a pointer where the result is to be placed. This is demonstrated in the following snippet.

```
// Set the specified 'name_store' to the names from the "a" resource.
void get_names_from_a(std::vector<string>& name_store);
```

Since C++11, it is generally encouraged to return vectors directly. This new style has several benefits, not least of which is reasonability of the code.

```
// Return the names from the "a" resource.
std::vector<string> get_names_from_a();
```

If the author happens to know a priori that several functions will be contributing to a vector *and* that high-performance is required, passing in a reference to the vector and contributing elements to the end may be preferred. Note, however, that we've lost the simple semantics in the above code and that the caller's usage as leaked into the function's interface.

```
// Get names from the "a" resource and add them onto the end of the specified
// 'name_store'.
void add_names_from_a(std::vector<int>& name_store);
```

The implementer may opt for a generic function instead if the compilation performance hit and extra complication are deemed worth the added flexibility.

```
// Get names from the "a" resource and assign them to subsequent iterators of
// the specified 'name_store'. The specified 'OutputIterator' must be an output
// iterator that accepts values of type 'std::string'.
template<typename OutputIterator>
void add_names_from_a(OutputIterator name_store);
```

However, most of the time the extra performance isn't needed and the extra genericity isn't warranted. What tops priorities is simplicity of specification and use.

```
// Return the names from the "a" resource.
```

```
std::vector<string> get_names_from_a();
```

The proposed `+` operator for `std::vector` is intended to support this kind of code where simplicity of implementation is the primary concern. When folks need higher performance, they can continue to make their interfaces more sophisticated to achieve it.

A model for `+` overloads

It may come as a surprise that we're using `+`, which is usually associated with a numeric sum operation, with a seemingly unrelated type, `std::vector`. There is precedent with overloading operators with meanings completely unrelated to the original semantics, as with C++ streams, but here we can generalize the original semantics without abandoning its essence.

`+` for numerics and `+` for `std::string` have in common certain properties. Namely, $a + (b + c) == (a + b) + c$ for all values `a`, `b`, and `c`. Operations like this with the types they apply to are called semigroups in mathematics. The proposed `+` operator for vectors also forms a semigroup. This concept can be used as a guide for when and where the `+` operator can be overloaded in a consistent way.

Note that all the aforementioned `+` overloads also have a zero element `e` where $e + a = a + e = a$ for all values `a`. Operations like this with the types they apply to are called monoids in mathematics. The default constructed values of all of numerics, `std::string`, and `std::vector` represent their corresponding zero elements with respect to `+`. Perhaps monoids would be an even better guide for overloading of the `+` operator.

Wording

If the committee deems this is worth pursuing, wording will be provided.

Conclusion

Lets keep simple things simple by allowing `+` to be used for concatenating vectors.